University of Geneva

Faculty of Science

Series 7: Genetic Programming

Metaheuristics for Optimization
14x013

Michel Jean Joseph Donnet

2025-12-29

Contents

1 Introduction

Technological advances in recent years have enabled humans to solve complex real-life problems through relevant problem modeling, allowing them to harness the power of computers. However, some concrete problems are too complex for a computer to find a solution in polynomial time.

For example, finding the analytical form of a function for which only certain output values are known requires checking different types of functions with different types of values, which necessitates searching an astronomical search space. This problem has practical applications in many fields, such as physics and economics.

This is why certain metaheuristic methods have been developed to deal with the complexity of this type of problem with a huge search space, providing a sufficiently good solution in polynomial time. One of the metaheuristic approaches, called genetic programming, involves imitating the evolution of populations in nature and applying it to the analytical form of a function in order to find an analytical form that corresponds to the result of the given function. The idea behind this concept is inspired by biology: let's assume we have an initial generation of individuals. The best individuals in the population are more likely to survive thanks to natural selection. This is called the selection stage. Next, the individuals reproduce, creating offspring that is a mixture of the parents' genetic material. Thanks to this mixture, the offspring may have better characteristics than their parents. This is called the crossover stage. In addition, the offspring have certain mutations that have altered their genetic material, making them better (or worse) than their parents. This stage is called mutation. Finally, the new generation will produce a new generation, and so on, which will improve certain characteristics that allow individuals to survive better. The genetic programming metaheuristic works in a similar way with generations: an initial generation of programs is created. Then, programs are selected with a higher chance of being selected for good programs during the selection stage. Next, programs are mixed in pairs during the crossover stage. Mutations are applied to the newly created programs, and the stages are repeated for the new generation, and so on.

This work will focus on a specific problem that deals only with Booleans and basic operations on them. A set of variables and operators is provided to construct the function, and the objective is to find a correct assignment of operators and variables that reproduces the given binary table. In our case, we only have 4 variables with 4 operators: "AND", "OR", "NOT", and "XOR". So the search space for the given problem corresponds to all possible combinations of these variables and operators, in accordance with the program length constraint: the combination must not exceed the maximum length specified.

To evaluate each program generated by the combination of operators and variables, the fitness function is defined. The fitness function is simply a comparison between the result generated by the program and the expected function: the fitness of a given program corresponds to the number of results that match the expected results. This means that the maximum fitness searched corresponds to the size of the dataset to be matched.

2 Methodology

(1.0)

- Initial Population
- selection
- crossover
- mutation
- validity ∗

3 Implementation

(1.5)

- What are you doing to do with the parameters ?
- Pseudo-code

4 Results

(1.0) Explored variations of parameters

5 Discussion

(2.0)

- Pm, Pc, Iterations
- Population Size, Tournament Size
- Program length, variable length (what must we change if program is not fixed size ??? theoretical… No need to implement it…)

6 Conclusion

(0.25)